

Project Setup 1

Fall 26-27 · Wed 09-16

Today you learn what you will build, start thinking about product ideas, and plan how your team will build the project skeleton.

Due

Date	What
Today, before you leave	Learning plan (<i>latest: Wed 09-23, start of class</i>)
Wed 09-30, start of class	Project candidates
Wed 10-07, start of class	Project skeleton

What You Will Build

Your team will design a product and build it over three sprints. **At the center of the product is an HTTP service with a documented interface, deployed on a free cloud tier.** Everything around that service is up to your team: who the product is for, what they do with it, and how they use it. You may build a client for it, such as a web app, a Discord bot, a mobile app or a game.

Build it for real people. Choose a group of people your team can describe and could actually talk to.

Aim high. Choose a product that needs your whole team for three sprints. You are not expected to build everything you plan. Deciding what to build first, and what to leave out, is part of the work.

There are three types of service. **Your instructor will assign your team one type.** Two teams will work on each type.

Aggregator

The service calls several outside services and combines what they return. The hard part: outside services are slow, limit how often you can call them, and sometimes fail. Your product must keep working when they do.

Examples:

- **Group trip planner.** Pulls flights, weather and local events into one itinerary that everyone in the group can edit.
- **Game-group Discord bot.** Collects each member's stats from several game platforms and posts weekly highlights.

- **Ruston civic feed.** Brings together city meeting agendas, weather alerts and road closures. People subscribe to what affects them.

Transformer

The service takes an input and produces something new from it. The hard part: some of the work takes too long to finish during a single request. Users must be able to start a job and come back for the result.

Examples:

- **Lecture to study guide.** Turns a lecture recording into a transcript, splits it into sections, and writes practice questions.
- **Sheet music from audio.** Turns a recorded melody into notation a musician can edit.
- **Receipts to budget.** Reads photos of receipts and turns them into categorized spending.

Ledger

The service records events and answers questions about them, including totals and summaries. The hard part: keeping the stored data correct, and keeping the interface stable while the way you store data changes.

Examples:

- **Intramural league manager.** Schedules, scores, standings and each player's history.
- **Shared household.** Chores, shared expenses and a grocery list, with a record of who did what.
- **Tabletop campaign tracker.** A D&D group's inventory, experience points and a log of every session.

Project Candidates — Due Wed 09-30

Submit at least two product ideas of your assigned type. For each, state who it is for, what it does, and why it needs your whole team for three sprints. At Setup 3 your instructor approves one or sends them back.

Project Skeleton and Learning Plan

The Project Skeleton — Due Wed 10-07

Before you choose a product, build a minimal service and deploy it. The skeleton is done when **someone outside your team can confirm each of these without asking you:**

1. Opening your service's public web address in a browser returns a response from your service.
2. The service has at least two endpoints (web addresses it answers). One of them, when sent a request it cannot handle, reports the failure in a way a program can detect without reading the message text.
3. A page at a fixed address lists every endpoint: what each one expects and what it returns.
4. At least two automated tests run every time code is pushed to your repository, and the repository shows whether they passed.

You may use any language, framework and host. A working skeleton can be about fifty lines of code. Working out how to build it is part of the assignment.

Cloud account. Google Cloud and AWS both require a credit card to open an account. **Azure for Students** does not: sign up with your school email to get \$100 of credit. Use Azure unless you already have another provider set up.

Example: A Dice Roller

This is an example of a finished skeleton, not a project idea. The service rolls one die with a chosen number of sides. `<address>` stands for the service's public web address.

1. The public address responds.

- **Send:** open `https://<address>/` in a browser.
- **Get back:**

```
{"service": "Dice Roller", "status": "running"}
```

2. Two endpoints, and one reports failure.

The first endpoint is `/`, shown above. The second is `/roll`.

- **Send:** `https://<address>/roll?sides=6`
- **Get back:** status code **200** (success) and

```
{"sides": 6, "result": 4}
```

- **Send:** `https://<address>/roll?sides=0`
- **Get back:** status code **400** (bad request) and

```
{"error": "sides must be a whole number from 2 to 100"}
```

A program checks the status code (200 or 400) to know whether the request worked. It never has to read the error message.

3. A page lists every endpoint.

- **Send:** open `https://<address>/docs` in a browser.
- **Get back:** a page showing:

Endpoint	Expects	Returns
<code>/</code>	nothing	the service name and "running"
<code>/roll</code>	sides: a whole number from 2 to 100	status 200 with the number of sides and the result, or status 400 with an error message

4. Two tests run on every push.

Test	Sends	Passes when
A valid roll works	/roll?sides=6	the status code is 200 and result is from 1 to 6
An invalid roll is refused	/roll?sides=0	the status code is 400

- **Result:** each push in the repository's history shows a pass or fail mark. Opening the mark shows which test failed.

The Learning Plan — Due Today

Submit your learning plan before you leave class today and start carrying it out. Wed 09-23 is the latest it will be accepted, for teams that cannot finish today because a member is absent.

Your team does not yet know how to build the skeleton. Plan how you will learn. The plan must state:

1. **What needs to be learned.** List each thing the skeleton requires that your team cannot do yet.
2. **Who learns each thing, from which sources, and by what date.**
3. **Who builds each part of the skeleton, who approves it, and by what date.**
4. **Backup.** For each part, name a second person who could finish it if the owner becomes unavailable, and say how that person will know enough to do so.
5. **Teaching the team.** Decide how and when everyone on the team will learn what each member has learned, so that no part of the skeleton is understood by only one person.